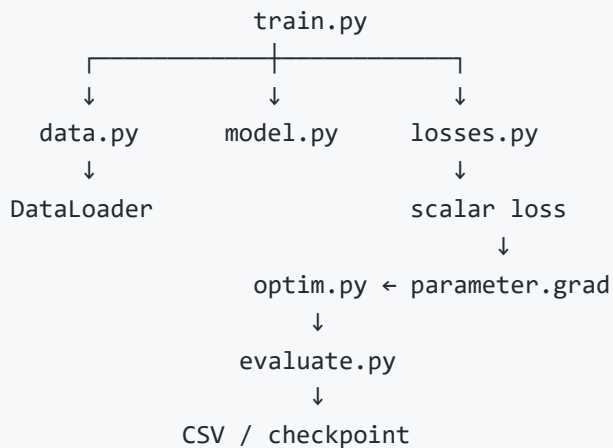


基于 CIFAR-10 数据集：从零搭建图像分类器

模型目的：训练模型，将图像分类为10种。项目通过手写loss反向传播，优化器，评估，模型文件，旨在复习基础知识，理解从零到一搭建一个模型的过程。该项目采用的数据集为Stanford CIFAR10。项目的一条“数据加工流水线”：

```
YAML 配置
↓
utils.py: 读取配置、固定随机性
↓
data.py: pickle 文件 → NumPy → Dataset → DataLoader
↓
train.py: 取 batch → 模型前向 → loss → 反向传播 → 参数更新
↓
evaluate.py: 关闭梯度 → 遍历评估集 → 汇总 loss / accuracy
↓
CSV 日志 + checkpoint
```

项目中 train.py 是总调度模块（composition root），它把其他模块连接起来：



在非手写的pytorch框架里，evaluate.py losses.py optim.py 直接集成在train.py里面

utils.py

该文件是基础设施的加载。

1. 为各模块设置seed，即设计随机性
2. 加载configs/.yaml文件里面的配置，将YAML的参数值转换成普通python字典，如超参数的值，batch_size的大小，hidden dimension的大小

data.py

首先我们要理解数据集是怎么整理数据的，然后再理解为什么需要data.py处理数据才能将数据拿来训练。

数据集

因为我们实现的是一个图片分类器，所以CIFAR-10 有十个类别：

```
0: airplane
1: automobile
2: bird
3: cat
4: deer
5: dog
6: frog
7: horse
8: ship
9: truck
```

共有：

```
训练集：50,000 张
测试集：10,000 张
总计： 60,000 张
```

每张图片：

```
宽度：32
高度：32
通道：3（R、G、B）
 $3 \times 32 \times 32 = 3072$  个像素通道值
用(3072,)的向量储存
前1024个元素储存R，中间1024个元素储存G，后面1024个元素储存B
```

数据集在磁盘组织形式是这样的：

```
cifar-10-batches-py/
├─ data_batch_1
├─ data_batch_2
├─ data_batch_3
├─ data_batch_4
├─ data_batch_5
├─ test_batch
└─ batches.meta
```

整个文件可以拆成三层：

一个数据集是这样储存的：

```

{
    "data": ndarray,          # (10000, 3072), uint8
    "labels": list,           # 10000 个整数
    "batch_label": str,
    "filenames": list,        # 10000 个文件名
}

#相当于:
batch["data"] = [
    [第0张图的3072个值],
    [第1张图的3072个值],
    ...
    [第9999张图的3072个值],
]

batch["labels"] = [
    0,
    6,
    0,
    2,
    ...
]

```

load_cifar10

加载原始数据，拼接为训练集/验证集/测试集。此处验证集为官方集

1. load函数：每次读取一个pickle文件，将其转化为python的词典

注意：pickle文件不是必须的，它只是一种序列化的图片表达方式。JPG，PNG文件也可以，只需要定义合适的Dataset类将其转化为符合pytorch训练范式的数据即可

```

def _load(fname):
    with open(os.path.join(cifar_dir, fname), "rb") as f:
        return pickle.load(f, encoding="bytes")

#过程是:
#data_batch_1 文件
#    ↓ open(..., "rb")
#二进制文件对象
#    ↓ pickle.load
#Python dict

```

这里pickle 不是普通图片对象，它保存的是序列化后的 Python 对象。

2. 拼接5个data batch得到训练集+加载测试集

```

    train_X, train_y = [], []
# 拼接
    for i in range(1, 6):

        batch = _load(f"data_batch_{i}")

        train_X.append(_field(batch, "data"))

        train_y.extend(_field(batch, "labels"))

    X_train = np.concatenate(train_X).astype(np.uint8)

    y_train = np.asarray(train_y, dtype=np.int64)


# 得到测试集
    test = _load("test_batch")

    X_test = _field(test, "data").astype(np.uint8)

    y_test = np.asarray(_field(test, "labels"), dtype=np.int64)

    return X_train, y_train, X_test, y_test

```

CIFAR10Dataset类

作用是把加载完毕的X_train,Y_train,X_test,Y_test规范为pytorch范式

一个 PyTorch Dataset 最重要的协议只有两个：

```

len(dataset)
dataset[index]

```

```

class CIFAR10Dataset(Dataset):

    # 初始化记录图片
    def __init__(self, images: np.ndarray, labels: np.ndarray,
                 mean, std, transform: Optional[Callable] = None):

        assert images.ndim == 2 and images.shape[1] == 3072, f"bad images shape {images.shape}"

        self.images = images

        self.labels = labels

        self.mean = torch.as_tensor(mean, dtype=torch.float32).view(3, 1, 1)

        self.std = torch.as_tensor(std, dtype=torch.float32).view(3, 1, 1)

        self.transform = transform

    # 定义len
    def __len__(self) -> int:

        return len(self.labels)

    # 定义获取一张图片的方法，将数据集的(3072, , )的向量进行处理，最终返回处理完毕的图片及其标签，可以用于训练
    def __getitem__(self, idx: int):

        # uint8 (3072,) -> float32 (3, 32, 32) in [0, 1]

        img = (

            torch.from_numpy(self.images[idx])# 加载

            .view(3, 32, 32) # 变换维度

            .to(torch.float32) # type

            .div_(255.0)# 缩放到[0,1]

        )

        if self.transform is not None:

            img = self.transform(img)# 数据增强，比如裁剪，旋转

        img = (img - self.mean) / self.std # per-channel normalize

```

```
return img, int(self.labels[idx])
```

在外调用时

```
train_set = CIFAR10Dataset(  
    images=X_train,  
    labels=y_train,  
    mean=[0.4914, 0.4822, 0.4465],  
    std=[0.2470, 0.2435, 0.2616],  
    transform=train_transform,  
)
```

get_dataloaders

集成上面的load函数和类，将dataset打包成batch

```

def get_dataloaders(cfg: dict) -> Tuple[DataLoader, DataLoader]:
    # 获取数据集
    X_train, y_train, X_test, y_test = load_cifar10(cfg.get("cifar_dir"))

    mean, std = cfg["mean"], cfg["std"]

    num_workers = cfg.get("num_workers", 0)

    # 是否采用数据增强
    train_transform = None

    if cfg.get("augment", False):

        train_transform = T.Compose([

            T.RandomCrop(32, padding=4),

            T.RandomHorizontalFlip(),

        ])

    # 将数据转化为pytorch范式
    train_set = CIFAR10Dataset(X_train, y_train, mean, std, transform=train_transform)

    test_set = CIFAR10Dataset(X_test, y_test, mean, std)

    # 将数据集打包为数据batch, DataLoader是一个类
    train_loader = DataLoader(

        train_set, batch_size=cfg["batch_size"], shuffle=True, # 从config词典里拿到每一批的大小, 创建批次。shuffle=True意味着打乱批次中图片的顺序

        num_workers=num_workers, drop_last=False, # 不丢弃余数

    )

    test_loader = DataLoader(

        test_set, batch_size=cfg["batch_size"], shuffle=False,

        num_workers=num_workers,

    )

    return train_loader, test_loader

```

Dataset 和 DataLoader 的区别

```
train_set[7]
# 返回单张图片的信息
image: (3, 32, 32)
label: int
```

```
next(iter(train_loader))
# 返回128张图片的批次信息
x: (128, 3, 32, 32)
y: (128,)
```

为什么要分mini-batch训练，而不是一张一张图片训练？

1. 如果全批量梯度下降（full-batch gradient descent） 假设训练集有 50,000 张图片，可以理论上这样做：

```
x.shape == (50000, 3, 32, 32)
y.shape == (50000,)

logits = model(x)
loss = loss_fn(logits, y)

loss.backward()
optimizer.step()
```

问题：会导致内存不够，计算反馈太慢

2. 如果 batch size 是 128：

```
输入: (128, 3, 32, 32)
第一层激活: (128, 32, 32, 32)
```

3. 如果一张一张训练，相当于batch_size = 1 由于计算出来的全批量梯度为

$$g = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L_i(\theta)$$

mini-batch梯度为

$$g = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\theta} L_i(\theta)$$

如果size=1：

- 每一次前向传播得到的结果g噪声很大
- 会导致擅长并行计算的GPU大部分处于空闲，浪费资源 所以mini-batch是折中方法

model.py

定义框架 定义model类型，使用nn.Module/nn.Sequential定义架构

写代码时注意维度的变化即可，在这里不多赘述。

train.py

在这里直接给出一般pytorch架构最核心的循环架构：

```

import torch
import torch.nn as nn

from cifar.data import get_dataloaders
from cifar.model import SmallCNN

def main():
    # 获取超参数
    config = {
        "cifar_dir": None,
        "mean": [0.4914, 0.4822, 0.4465],
        "std": [0.2470, 0.2435, 0.2616],
        "batch_size": 128,
        "augment": True,
        "num_workers": 0,
    }
    num_epochs = 10
    learning_rate = 1e-3

    # 设备
    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )
    print(f"Using device: {device}")

    # 获取batch数据
    train_loader, val_loader = get_dataloaders(config)

    # 获取模型
    model = build_model(..)
    # 定义loss计算方式
    criterion = nn.CrossEntropyLoss()
    # 获取优化器
    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=learning_rate,
    )

    best_val_acc = 0.0
    # 训练循环, epoch规定循环次数
    for epoch in range(num_epochs):
        # 训练模式
        model.train()

        train_loss_sum = 0.0
        train_correct = 0
        train_total = 0

```

```

for x, y in train_loader:
    x = x.to(device) # train batch
    y = y.to(device) # labels batch
    # 清理梯度
    optimizer.zero_grad(set_to_none=True)

    # 前向
    logits = model(x)

    # 计算loss
    loss = criterion(logits, y)

    # 反向传播
    loss.backward()

    # 更新参数
    optimizer.step()

    # 统计训练指标
    batch_size = x.size(0)

    train_loss_sum += loss.item() * batch_size
    train_correct += (
        logits.argmax(dim=1) == y
    ).sum().item()
    train_total += batch_size

train_loss = train_loss_sum / train_total
train_acc = train_correct / train_total

# 验证模式
model.eval()

val_loss_sum = 0.0
val_correct = 0
val_total = 0
# 对于验证模式，不需要反向传播，不更新模型参数，所以开启推理模式，不记录梯度
with torch.inference_mode():
    for x, y in val_loader:
        x = x.to(device)
        y = y.to(device)

        logits = model(x)
        loss = criterion(logits, y)

        batch_size = x.size(0)

        val_loss_sum += loss.item() * batch_size

```

```

        val_correct += (
            logits.argmax(dim=1) == y
        ).sum().item()
        val_total += batch_size

    val_loss = val_loss_sum / val_total
    val_acc = val_correct / val_total

    # 记录打印epoch的Loss和accuracy
    print(
        f"Epoch {epoch + 1:02d}/{num_epochs} | "
        f"train_loss={train_loss:.4f} | "
        f"train_acc={train_acc:.4f} | "
        f"val_loss={val_loss:.4f} | "
        f"val_acc={val_acc:.4f}"
    )
    # 记录最好模型
    if val_acc > best_val_acc:
        best_val_acc = val_acc

        torch.save(
            model.state_dict(),
            "best_model.pt",
        )

    print(f"Training finished. Best val acc: {best_val_acc:.4f}")

if __name__ == "__main__":
    main()

```

train模式：

- Dropout 随机屏蔽部分神经元；
- BatchNorm 使用当前 batch 的均值和方差；
- BatchNorm 更新运行均值和方差（running statistics）；
- Linear、Conv2d、ReLU 等普通层通常不受影响。 validation模式：
- Dropout 停止随机屏蔽，相当于恒等映射；
- BatchNorm 使用训练时积累的运行均值和方差；
- BatchNorm 不再被验证数据更新。

训练结果保存

CSV 日志

CSV 是一种表格文本格式：

```
epoch,train_loss,train_acc,val_loss,val_acc,time_s
1,1.6215,0.4031,1.3168,0.5357,30.94
2,1.3153,0.5270,1.0914,0.6063,29.06
3,1.1946,0.5713,1.0786,0.6072,28.87
```

每一行记录一个 epoch 的结果

CSV 如何写入

训练开始时建立表头：

```
import csv

csv_file = open("training_log.csv", "w", newline="")
writer = csv.writer(csv_file)

writer.writerow([
    "epoch",
    "train_loss",
    "train_acc",
    "val_loss",
    "val_acc",
])
```

每个 epoch 结束后写一行：

```
writer.writerow([
    epoch,
    train_loss,
    train_acc,
    val_loss,
    val_acc,
])
```

最后关闭：

```
csv_file.close()
```

checkpoint

checkpoint 是模型训练状态的存档文件，通常扩展名是：

```
.pt  
.pth  
.ckpt
```

checkpoint 通常保存一个 Python 字典:

```
checkpoint = {  
    "epoch": epoch,  
    "model": model.state_dict(),  
    "optimizer": optimizer.state_dict(),  
    "best_acc": best_acc,  
    "cfg": config,  
}
```

然后:

```
torch.save(checkpoint, "checkpoint.pt")
```

如何从 checkpoint 恢复训练

```
checkpoint = torch.load(  
    "last.pt",  
    map_location=device,  
)  
  
model.load_state_dict(checkpoint["model"])  
optimizer.load_state_dict(checkpoint["optimizer"])  
  
start_epoch = checkpoint["epoch"] + 1  
best_acc = checkpoint["best_acc"]  
  
for epoch in range(start_epoch, num_epochs):  
    train code
```